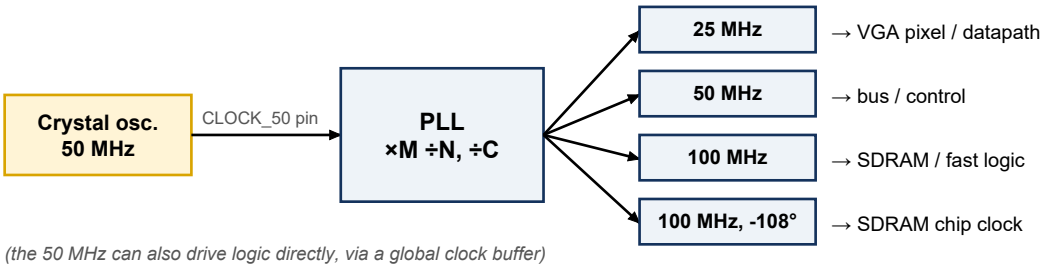
Clocks & PLLs in FPGAs

Where clocks come from, how a PLL makes 25 / 50 / 100 MHz from one crystal, how to cross between them safely, and how to constrain them. Examples use the Cyclone V SoC on the DE10-Standard and Quartus Prime.

1. The big picture: why one board can run at 25, 50 and 100 MHz

A board does not contain a separate crystal for every speed you see in a design. It carries a few fixed oscillators (on the DE10-Standard, several 50 MHz ones, plus 25 MHz references for the HPS and some peripherals). Inside the FPGA, **PLLs** (phase-locked loops) multiply and divide a reference clock to generate as many related clocks as the design needs. So "the same board can use 25, 50 and 100 MHz" really means: one 50 MHz input, then a PLL producing 25 MHz (÷2 of 50), 50 MHz (same) and 100 MHz (×2) at the same time, each with its own frequency, phase and duty cycle.



(the 50 MHz can also drive logic directly, via a global clock buffer)

> **Key idea.** Every clock in a design is either (a) a physical input from a crystal/oscillator/other chip, (b) the output of a PLL (or other clock-management block) fed from one of those inputs, or (c) an illegal-ish creation of your own logic (a divider, a gated clock). Prefer (a) and (b); treat (c) as a last resort.

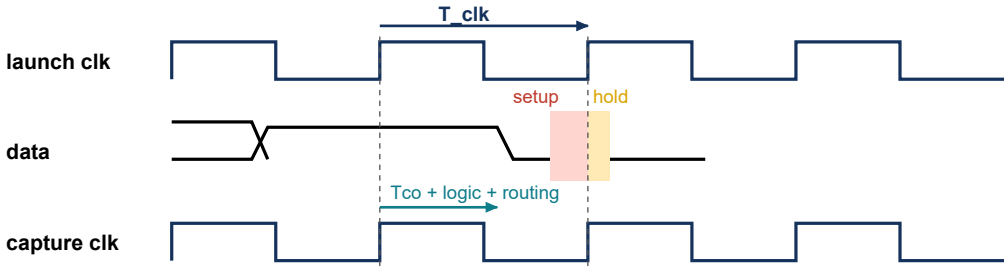
Way to get a new frequency	How	Good for	Cost / risk
Use the oscillator directly	Pin → global clock buffer → flip-flops	The base rate of your design	Frequency is fixed by the board
PLL	Multiply by M/N, divide by C; adjustable phase	Any frequency; low jitter; several outputs; phase shifting; clean duty cycle	Limited number of PLLs; needs lock time; M, N, C ranges limit the ratios
Clock enable (a pulse, not a clock)	Counter makes a 1-cycle *enable* every k-th clock; flops use `if (en)`	Slow logic (UART, I2C, LEDs, 1 Hz tick) in the same clock domain	Needs no clock-domain crossing; timing is still analysed at the fast clock
Fabric clock divider (flop output used as clock)	`always @(posedge div)`	Almost never	Creates a new clock domain, bad skew, needs extra constraints. Avoid.

2. Clock sources on a board (DE10-Standard)

- **Crystal / MEMS oscillator.** Fixed frequency, parts-per-million accuracy, very low jitter. The DE10-Standard provides 50 MHz inputs on dedicated clock pins (`CLOCK_50`, `CLOCK2_50`, `CLOCK3_50`, and a fourth wired to the HPS side). The HPS (ARM) has its own 25 MHz reference for its internal PLLs, which generate the CPU, DDR3 and peripheral clocks.
- **Clock-capable pins.** Only certain pins (`CLK[n]p/n`) can drive the clock networks and PLL reference inputs directly. A clock sent to an ordinary I/O pin has to go through general routing and arrives with large skew.
- **Clock generators / jitter cleaners** (e.g. Si5338 on many boards). They are programmable chips that produce several frequencies from one crystal, usually over I2C. Same idea as an FPGA PLL, but external.
- **Recovered or forwarded clocks.** A video decoder, Ethernet PHY or ADC supplies its own clock with its data (pixel clock, RX clock, sample clock). It is asynchronous to your oscillator, so it is its own clock domain.
- **The board's own pin list is the truth.** Check the user manual / golden top for which pin is which clock and what voltage standard it uses.

3. Why the clock is the heart of timing

A synchronous design is a chain of flip-flops with combinational logic between them. At every active clock edge the launching flop sends out new data; the capturing flop must see that data stable **around the next edge**. Two requirements follow:



- **Setup:** data must arrive at least T_{su} *before* the capture edge. $T_{clk} \geq T_{co} + T_{logic} + T_{routing} + T_{su}$ + skew + uncertainty. This limits **maximum frequency** (F_{max}). Fix by pipelining, simpler logic, or a slower clock.
- **Hold:** data must stay stable T_h *after* the capture edge, so the new data must not race through too soon. This does not depend on frequency; the tools fix it by adding delay. Large clock skew causes hold failures.
- **Slack** = required time − arrival time. Positive means met, negative means failing. The Timing Analyzer reports setup and hold slack per clock, per corner (slow 85 °C, slow 0 °C, fast...).
- $F_{max} \approx 1 / (T_{clk}$ − worst setup slack). Example: 25 MHz (40 ns) with +12 ns slack means the worst path needs 28 ns, $F_{max} \approx 35.7$ MHz.

Rule of thumb. Raising the clock only shrinks setup slack, never hold slack. If hold fails, the cause is clock skew or a crossing between clock domains, not that the clock is too fast.

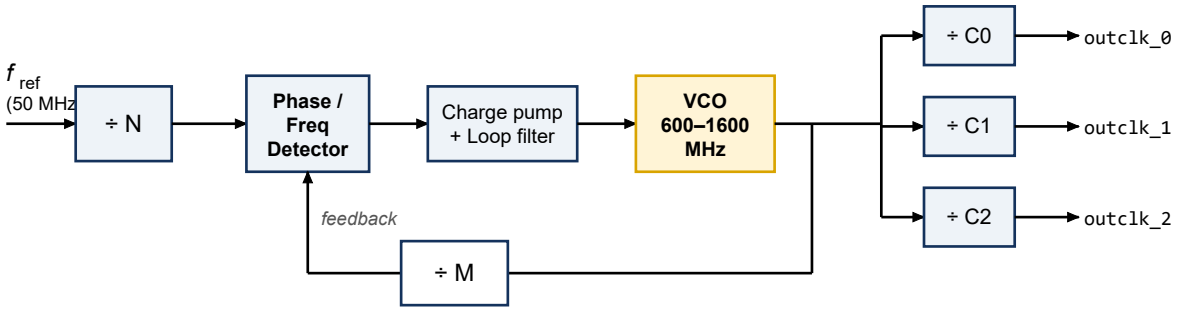
4. Clock networks inside the FPGA

A clock has to reach tens of thousands of flip-flops at almost the same time. FPGAs therefore have dedicated, low-skew, high-fanout clock networks, separate from ordinary routing:

Network (Cyclone V naming)	Reach	Typical use
Global clock (GCLK)	Whole device; about 16 networks	Main clocks with large fanout. Each PLL output you use as a clock normally gets one.
Regional clock (RCLK)	One quadrant / region	Clocks with moderate fanout; saves global resources.
Periphery clock (PCLK)	I/O banks / transceiver side	High-speed I/O and DDR interfaces.

- Quartus automatically inserts a **clock control block** (buffer) on each clock and promotes high-fanout signals to global networks. The number of global networks is a hard budget: 3 PLL outputs + 2 external clocks + a few promoted resets can use up several.
- Skew between flops on the same global network is small and is already counted by the timing tool. Skew between *different* clocks is not reduced, which is why unrelated or poorly aligned clocks need CDC logic.
- A **clock gate** (clock enable on the clock control block) can switch a whole network off to save power. Use the vendor primitive (`altclkctrl`), never `assign gclk = clk & en;`.

5. How a PLL works



f_VCO = f_ref × M / N **f_out(i) = f_VCO / C_i** **PFD frequency = f_ref / N (must be in the allowed range)**

- **Phase/frequency detector (PFD):** compares the reference (after ÷N) with the feedback (VCO ÷ M) and produces up/down pulses proportional to the phase error.
- **Charge pump + loop filter:** turn those pulses into a smooth control voltage. Filter bandwidth sets how fast the PLL locks and how much input jitter it filters.
- **VCO (voltage-controlled oscillator):** its frequency follows the control voltage, kept inside a legal band (roughly 600–1600 MHz on Cyclone V; check the datasheet for your speed grade).
- **Feedback divider ÷M:** forces VCO/M to equal f_{ref}/N once locked, so $\mathbf{f_{VCO} = f_{ref} \cdot M / N}$.
- **Post-scale counters ÷C_i:** each output has its own divider from the same VCO, so all outputs are phase-aligned multiples of the VCO. This is how one PLL gives 25, 50 and 100 MHz together.
- **Fractional PLL (Cyclone V fPLL):** M can have a fractional part (delta-sigma modulation), allowing ratios like 14.85 that integer M/N cannot hit with a reasonable PFD frequency.

Things a PLL gives you that dividers do not

Feature	Meaning	Why it matters
Frequency synthesis	Any $f_{out} = f_{ref} \cdot M/(N \cdot C)$	Pixel clocks, SDRAM clocks, audio clocks
Phase shift	Each output shifted by a fraction of the VCO period (resolution about $T_{VCO}/8$)	SDRAM clock leading/lagging data, source-synchronous capture
Duty cycle control	Programmable high/low ratio per output	Rarely changed; 50 % is the default
Jitter filtering / clean-up	Loop filter smooths input noise	Cleaner clocks than a fabric divider ever gives
Clock network compensation	Feedback taken from a clock tree so that tree delay is cancelled	Aligning internal clocks with an external pin
`locked` output	High when the loop is stable	Hold the design in reset until it is high

> **Key idea.** Lock takes microseconds. Until `locked` is high the output clock may be wrong, glitchy or absent, so **never release your design reset before locked** (and synchronise it, see section 9).

6. Choosing PLL settings: worked examples

Procedure: pick the reference (50 MHz), find a VCO frequency in range that is an integer multiple of every output you need, then choose M, N and C_i. In practice the Quartus PLL IP does this for you; knowing the arithmetic lets you sanity-check it and understand its errors.

Wanted	N	M	VCO	C	Result	Comment
25, 50, 100, 200 MHz	1	16	50×16 = 800 MHz	32, 16, 8, 4	Exact	One VCO serves all of them; simplest case
74.25 and 148.5 MHz (720p / 1080p pixel clock)	1	14.85 (fractional)	742.5 MHz	10, 5	Exact	Integer M/N would need PFD = 2.5 MHz, too low; fractional mode fixes it
25.175 MHz (true VGA 640×480)	1	16.112 (fractional)	805.6 MHz	32	Exact in fractional mode	25.0 MHz also works on nearly every monitor; error 0.7 %

Wanted	N	M	VCO	C	Result	Comment
100 MHz, phase −108°	1	16	800 MHz	8	Exact	−108° of 10 ns = −3 ns; for SDRAM clock-out

Common target frequencies

Frequency	Used for
25.175 (25) MHz	VGA 640×480 @ 60 Hz pixel clock
40 / 65 / 108 MHz	SVGA 800×600 / XGA 1024×768 / SXGA 1280×1024 pixel clock (all 60 Hz)
74.25 / 148.5 MHz	HD 1280×720 / Full HD 1920×1080 @ 60 Hz pixel clock
50 MHz	RMII Ethernet reference; general system/bus clock
100 / 133 / 143 MHz	SDR SDRAM interface
125 MHz	Gigabit Ethernet (GMII / RGMII)
12.288 / 11.2896 MHz	Audio master clock (256 × 48 kHz / 256 × 44.1 kHz)
60 MHz	USB ULPI PHY clock
Up to 400 MHz and beyond	DDR3 memory clocks (handled by the memory controller PLLs, not hand-built)

Rule of thumb. Keep f_{ref}/N (the PFD frequency) as high as allowed and N small (usually 1). Higher PFD frequency gives lower in-band noise and faster lock. Keep the VCO in the middle of its range when you have a choice.

7. Using the PLL in Quartus (Cyclone V)

- Open **IP Catalog** → **PLL Intel FPGA IP** (called *Altera PLL* in older versions; *ALTPLL* is the legacy megafunction). Choose the device family, **reference clock frequency = 50 MHz** and operation mode (*normal*, *direct*, *source synchronous*, *zero delay buffer*).
- Choose **integer-N** or **fractional-N** mode, number of output clocks, and for each: desired frequency, phase shift, duty cycle. The tool shows the *actual* frequency it can reach; read this, not your wish.
- Optional ports: `rst` (reset the PLL), `locked` (use it), dynamic reconfiguration (change M/N/C at runtime), dynamic phase shift.
- Generate HDL, add the `.qip` to the project and instantiate:

```
// 50 MHz in -> 25, 50, 100 MHz out
pll_main u_pll (
    .refclk   (CLOCK_50),   // reference, from a dedicated clock pin
    .rst      (1'b0),       // PLL reset; tie low if unused
    .outclk_0 (clk25),
    .outclk_1 (clk50),
    .outclk_2 (clk100),
    .locked   (pll_locked)
);
```

Reset that follows the clock: async assert, sync release

```
// one copy per clock domain
module reset_sync (input clk, input arst_n, output rst_n);
    reg [1:0] q;
    always @(posedge clk or negedge arst_n)
        if (!arst_n) q <= 2'b00;
        else         q <= {q[0], 1'b1};
    assign rst_n = q[1];
endmodule

// arst_n = KEY[0] & pll_locked  -> design stays in reset until the PLL is stable
```

Watch out. The reset input is also a signal crossing into the domain. If it is released asynchronously, some flops see it release in one clock cycle and others in the next (reset recovery/removal violation). The synchronizer above removes this; it is also why the PLL `locked` signal belongs *in the reset path*.

8. Dividers, clock enables and clock gating: design rules

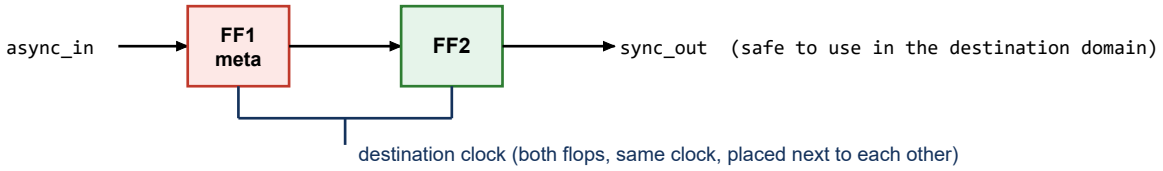
```
// BAD: derived clock from fabric logic
always @(posedge clk100) div <= ~div;
always @(posedge div)    x   <= y;        // new clock domain, needs generated-clock constraint,
                                          // skew vs clk100 uncontrolled

// GOOD: stay in one clock, use an enable
reg [1:0] cnt;
always @(posedge clk100) cnt <= cnt + 2'd1;
wire en25 = (cnt == 2'd0);                // one-cycle pulse every 4 clocks = 25 MHz rate
always @(posedge clk100) if (en25) x <= y;
```

- **Enable beats divided clock** whenever the slow part and the fast part talk to each other. Everything stays in one clock domain, so no synchronizers are needed, and the timing tool checks one clock.
- If the slow signal truly must be an external clock (e.g. the pixel clock for a DAC), generate it with a **PLL output**, or forward it through a **DDR output register** (section 11), not a plain flip-flop output.
- Do not gate a clock with an AND/OR gate: glitches create false edges. Use the clock control block or an enable.
- Do not feed a clock into a data input or a data signal into a clock input without understanding that the tool then treats it as unconstrained or as a new clock.
- Aim for as few clock domains as the system truly needs. Each additional domain adds crossings, constraints and bugs.

9. Multiple clock domains and clock-domain crossing (CDC)

When data goes from flops on clock A to flops on clock B, the capture flop can sample while the signal is changing. It then goes **metastable**: its output hovers between 0 and 1 for an unknown time and can corrupt downstream logic. Metastability cannot be eliminated, only made rare enough (MTBF of centuries) by giving it time to resolve.



What crosses	Technique	Notes
Single bit, level that stays high ≥ 2 destination clocks (status, enable)	2-flop synchronizer (above)	Destination must see the level for at least about 1.5 destination periods. Add a third flop above ~200–300 MHz.
Single-cycle pulse, fast to slow	Toggle in the source, 2-FF synchronizer, XOR edge detect in the destination	A direct pulse can be missed entirely when the destination is slower.
Multi-bit value that changes rarely (config registers)	Handshake (req/ack, each synchronized)	Hold data stable until ack is seen; data bus is not synchronized bit by bit.
Multi-bit stream (pixels, samples)	Asynchronous (dual-clock) FIFO, e.g. DCFIFO / FIFO Intel IP	Gray-coded pointers cross, data memory is the buffer. Use the vendor IP.
Counter/pointer value	Gray code, then synchronizer	Only one bit changes per step, so a stale read is off by one, not garbage.
Related clocks from the same PLL (25 / 50 / 100 MHz)	Synchronous crossing, if the tool times it	Edges are aligned, so Quartus can analyse the path (the launch/capture pair is the nearest edges). Check slack on it. No synchronizer is needed if timing is met, but enables are safer.

Watch out. Never synchronise each bit of a multi-bit bus separately. Different bits settle at different times, so you can read a value that never existed (e.g. 0111 → 1000 seen as 1111). Use a FIFO, a handshake, or Gray code.

Rule of thumb. Clocks from the *same PLL* are related; clocks from different oscillators or from external devices are *asynchronous*. Treat the latter as true CDC. Treat the former as ordinary synchronous paths and let the timing analyzer check them.

10. Constraining clocks (SDC / Timing Analyzer)

The timing tool only checks what you tell it about. An unconstrained clock is not analysed at all, which means a design can "pass" with no timing check. Constraints live in a `.sdc` file added to the project:

```
# 1. Every board clock input
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]

# 2. Clocks made by PLLs: derived automatically from the PLL settings
derive_pll_clocks

# 3. Add the jitter/uncertainty margin
derive_clock_uncertainty

# 4. Truly unrelated clocks (e.g. an external pixel clock from a camera/decoder):
#    do not time paths between the groups. CLOCK_50 and its PLL outputs are one group.
create_clock -name EXT_PCLK -period 25.000 [get_ports EXT_PCLK]
set_clock_groups -asynchronous \
    -group [get_clocks {CLOCK_50 u_pll|*}] \
    -group [get_clocks EXT_PCLK]

# 5. Inputs/outputs relative to a clock (board-level timing)
set_input_delay  -clock CLOCK_50 -max 3.0 [get_ports DATA_IN*]
set_input_delay  -clock CLOCK_50 -min 0.5 [get_ports DATA_IN*]
set_output_delay -clock clk25    -max 2.0 [get_ports VGA_R*]

# 6. Slow human inputs: no timing meaning
set_false_path -from [get_ports {KEY* SW*}]
```

Command	What it does	Common mistake
`create_clock`	Defines a clock from a pin/net with its period	Wrong period (50 MHz = 20 ns, not 50)
`derive_pll_clocks`	Creates a clock for every PLL output from the IP settings	Forgetting it: PLL domains are then unconstrained
`create_generated_clock`	Declares a clock made from another by logic/divider	Missing for fabric-divided clocks and for forwarded output clocks
`set_clock_groups -asynchronous`	Stops analysis between listed clock groups	Using it on clocks that *are* related, hiding real violations
`set_false_path`	Removes a path from analysis	Using it to silence a real failing path
`set_multicycle_path`	Allows N cycles for a path (needs an enable that really spaces updates)	Forgetting the matching hold setting (N−1)
`set_input/output_delay`	Describes external timing at I/Os	Omitted: I/Os unconstrained

Reading the results

- **Clocks summary:** confirm that every expected clock appears (the board crystal *and* each PLL output). A missing PLL clock means the SDC lacks `derive_pll_clocks`.
- **Fmax summary / Slack per clock, per corner:** look at the worst corner (slow, 85 °C). Setup and hold both have to be ≥ 0.
- **Report Unconstrained Paths** and **Report Clock Transfers** find missing constraints and unexpected crossings. The **Clock Domain Crossing Viewer** in newer Quartus lists CDC paths and their synchronizers.
- "Design is not fully constrained" is a warning you should read, not ignore.

11. Clocks leaving or entering the chip (I/O timing)

- **Forwarded clock to a chip** (VGA DAC clock, SDRAM clock, camera XCLK): drive the output pin from a **DDR output register** (ALTDDIO_OUT / GPIO IP) with constant 1 and 0 inputs, or directly from the PLL output where the pin is a PLL output pin. A fabric flop output used as a clock has uncontrolled delay and jitter.
- **Source-synchronous inputs** (data arrives with its own clock, e.g. camera pixel clock, RGMII RX): use the incoming clock as the capture clock; constrain it with `create_clock` and set input delays against it.
- **Phase shift to centre the data eye:** use a PLL phase offset so the capturing edge falls in the middle of the valid data window. SDRAM-style example: data and address change after the rising edge of a 100 MHz clock; the SDRAM clock is shifted −3 ns (−108°) so its rising edge is in the middle of the data window.
- The output-delay numbers in the SDC should come from the receiving chip's datasheet (setup/hold) plus board trace delay, not guesses.

12. Jitter, skew, uncertainty, duty cycle

Term	Definition	Effect
Period jitter / cycle-to-cycle	Edge-to-edge variation of a clock around its ideal position	Eats timing margin; matters for ADC/DAC and fast I/O
Clock skew	Difference in clock arrival time between two flops	Positive skew helps setup and hurts hold; the tool includes it automatically for same-network flops
Clock uncertainty	Margin the tool subtracts for jitter, PLL phase error, and inter-clock skew	Added by `derive_clock_uncertainty`
Duty cycle	High time ÷ period	Matters for DDR and for min-pulse-width checks; PLL outputs hold 50 % well
PPM	Frequency tolerance of a crystal (e.g. ±50 ppm)	Two nominally equal oscillators drift apart, so they are asynchronous
Lock time	Time for the PLL to settle after power-up or reset	Hold the design in reset until locked

13. Common clock mistakes and a debugging checklist

Symptom	Likely clock cause	Check / fix
Works in simulation, dead on board	Reset released before PLL lock; or clock pin/IO standard wrong	Gate reset with `locked`; check pin location and voltage standard
Random glitches, rare data errors	Unsynchronised CDC, multi-bit bus crossing	Find every crossing; use sync / FIFO / handshake; run the CDC viewer
Timing passes but board fails	Missing clock in the SDC; unconstrained I/O; false paths set too broadly	Check the Clocks summary, Unconstrained Paths report, review every false path / clock group
Hold violations everywhere	Derived/gated clocks, or skew between domains	Replace with enables or a PLL; check clock network assignments
Output clock to a chip is unstable	Clock forwarded from a fabric flop	Use a DDR output register or PLL output
PLL never locks	Reference missing or outside the allowed range; PLL held in reset; bad M/N/C	Probe CLOCK_50; check IP settings; look at `locked` on an LED or SignalTap
Out of global clock networks	Too many clocks or promoted signals	Reduce clock count; turn off global promotion for non-clock nets
Monitor shows no sync, or an unstable image	Pixel clock not what the monitor expects	Verify the PLL's *actual* frequency; check blanking/sync widths

- Put `locked` and a heartbeat counter for each clock on LEDs. A blinking LED proves the clock is running at roughly the right speed: a 25-bit counter on 50 MHz toggles its MSB at about 0.75 Hz.
- When using SignalTap, clock the probe from the same domain as the signals it samples.

14. In this project (Adaptive Night-Vision on DE10-Standard)

- **Reference:** `CLOCK_50`. **Datapath clock:** 25 MHz from a PLL output (`divclk`), which is also the VGA pixel clock and `VGA_CLK` for the ADV7123 DAC. Running the whole datapath at the pixel clock gives lockstep 1 pixel per clock.
- **Reset:** `KEY[0] & pll_locked` goes through `reset_sync`, so the design is held in reset until the PLL is stable.
- **STA rule:** every build must show the PLL clock in the Timing Analyzer tables. The check script fails if it is absent. That is the "missing `derive_pll_clocks`" trap in section 10.
- **Open item:** VGA output timing against the DAC is not yet constrained (data and VGA_CLK come from the same PLL clock). If the image shows edge artefacts, add `set_output_delay` and consider a phase-shifted VGA_CLK output (section 11).
- **HPS domain:** the ARM side and DDR3 run on their own PLLs from the 25 MHz HPS reference. Anything crossing between the HPS bridges and the 25 MHz datapath is a true clock-domain crossing; use dual-clock FIFOs or handshakes (Platform Designer clock-crossing bridges/adapters provide these).

15. Cheat sheet and self-test

Formulas

$f_{VCO} = f_{ref} \cdot M / N$ | $f_{out} = f_{VCO} / C$ | $f_{PFD} = f_{ref} / N$

$T = 1/f$ (50 MHz = 20 ns, 100 MHz = 10 ns, 25 MHz = 40 ns) | phase shift (ns) = degrees/360 · T_{out}

Setup slack = $T_{clk} - (T_{co} + T_{logic} + T_{route} + T_{su}$ + skew + uncertainty) | $F_{max} \approx 1/(T_{clk} -$ slack)

Gray code: g = b ^ (b >> 1) | MTBF grows exponentially with each added synchronizer stage

Design rules

1) Use PLL or enable, not a fabric-divided clock. 2) Gate reset with `locked` and synchronise its release. 3) Constrain every clock; check the Clocks summary. 4) Every domain crossing has a named technique. 5) Forward clocks through DDR output registers or PLL outputs. 6) Fewer clock domains means fewer bugs.

Self-test (answers below)

1. A board has a 50 MHz oscillator. Give N, M and C values to produce 25 MHz and 100 MHz from the same PLL, with the VCO in 600–1600 MHz.
2. Why can an integer-N PLL not produce 74.25 MHz well from 50 MHz, and what fixes it?
3. A pulse in the 100 MHz domain is one clock wide. Why can a 2-FF synchronizer in a 25 MHz domain miss it, and what do you do?
4. Why is `assign gclk = clk & en;` a bad idea?
5. Setup fails by −1.2 ns at 100 MHz. List two ways to fix it. Does lowering the clock fix a hold violation?
6. What is wrong with the SDC that has `create_clock` for CLOCK_50 but no `derive_pll_clocks`?
7. Why must PLL `locked` be used in the reset path?
8. Two clocks come from the same PLL (25 MHz and 100 MHz). Are they asynchronous? What does that imply for the SDC?

Answers

1. N = 1, M = 16 gives VCO = 800 MHz; $C_0 = 32 \rightarrow 25$ MHz, $C_1 = 8 \rightarrow 100$ MHz.
2. It needs M/N = 14.85 = 297/20, which means a 2.5 MHz PFD, too low for good lock/jitter and possibly outside the legal range. Fractional-N PLL mode allows M = 14.85 directly with N = 1.
3. The pulse (10 ns) is shorter than the 40 ns destination period and may fall between edges. Toggle a flop in the source on each pulse, synchronize that level, and XOR consecutive samples to regenerate the pulse (or use a handshake / FIFO).
4. Gating with combinational logic can glitch whenever en changes while clk is high, creating false clock edges. Use a clock enable on the flops or the vendor clock-control block.
5. Pipeline the long path or reduce logic depth/fanout; or lower the clock until the period is at least 10 + 1.2 = 11.2 ns (about 89 MHz). No: hold slack does not depend on the clock period, so a hold violation needs a skew or crossing fix.
6. The PLL output clocks are not defined, so everything in those domains is unconstrained and no timing check happens on them. The Clocks summary will list only CLOCK_50.
7. Until locked, the output clock is unstable or missing. Releasing the design from reset then starts it in an undefined state. Its release also needs to be synchronised to avoid reset recovery violations.
8. No, they are related with aligned edges, so it is a synchronous crossing and the tool times it. Do not put them in an asynchronous clock group unless every crossing has a CDC structure; check the slack on that path instead.

Glossary

Term	Meaning	Term	Meaning
PLL	Phase-locked loop; locks an output to a multiple of the input	fPLL	Cyclone V fractional PLL
VCO	Voltage-controlled oscillator inside the PLL	PFD	Phase/frequency detector
GCLK / RCLK	Global / regional clock network	CDC	Clock-domain crossing
SDC	Synopsys Design Constraints, the timing constraint format	STA	Static timing analysis
Slack	Timing margin; negative means failure	Metastability	Flop output stuck between 0 and 1
DDIO	Double-data-rate I/O register (two bits per clock)	Gray code	Counting code, one bit changes per step
Fmax	Maximum clock rate meeting setup	Clock enable	Pulse that allows a flop to update

Device-specific numbers (VCO range, counter limits, number of PLLs and clock networks) are quoted approximately from memory of the Cyclone V family and can vary with speed grade. The Cyclone V device datasheet and the PLL IP parameter editor give the exact limits for the 5CSXFC6D6F31C6.